Question 1: Convert the Date Format

1. Aim

To write a menu-driven C program using a separate user-defined function to convert a date from `dd/mm/yyyy` format to `dd-mmm-yy` format.

2. Algorithm

1. Start the program.

2. Read a date in `dd/mm/yyyy` format.

3. Separate the day, month, and year.

4. Convert the numeric month into its three-letter abbreviation.

5. Extract the last two digits of the year.

6. Display the date in `dd-mmm-yy` format.

7. Stop the program.

3. C Program

```
#include <stdio.h>
#include <string.h>

#define SIZE 20

void convertDate(char date[]);

int main()
{
```

```
    char date[SIZE];

    printf("Enter date (dd/mm/yyyy): ");
    scanf("%19s", date);

    convertDate(date);

    return 0;
}

void convertDate(char date[])
{
    int day, month, year;
    char *months[] = {
        "Jan", "Feb", "Mar", "Apr",
        "May", "Jun", "Jul", "Aug",
        "Sep", "Oct", "Nov", "Dec"
    };

    if (sscanf(date, "%d/%d/%d", &day, &month, &year) != 3 ||
        day < 1 || day > 31 || month < 1 || month > 12 ||
        year < 0 || year > 9999)
    {
        printf("Invalid date format!\n");
        return;
    }

    printf("Converted date: %02d-%s-%02d\n",
           day, months[month - 1], year % 100);
}
```

4. Step-by-step explanation

Step 1: Read the date

`scanf("%19s", date);`
This reads the date entered by the user, such as `12/10/2026`.

Step 2: Separate the date components

`sscanf(date, "%d/%d/%d", &day, &month, &year);`
The `sscanf()` function extracts three integers:

- `day = 12`

- `month = 10`

- `year = 2026`

Step 3: Store month abbreviations

```
char *months[] = {
    "Jan", "Feb", "Mar", "Apr",
    "May", "Jun", "Jul", "Aug",
    "Sep", "Oct", "Nov", "Dec"
};
```

The month names are stored in an array. Since array indexing starts at zero, October is accessed using `months[9]`.

Step 4: Extract the last two digits of the year

```
year % 100
```

For `2026`, the result is `26`.

Step 5: Display the converted date

```
printf("Converted date: %02d-%s-%02d\n",
       day, months[month - 1], year % 100);
```

`%02d` prints the number using at least two digits, adding a leading zero when necessary.

5. Sample Output

```
Enter date (dd/mm/yyyy): 12/10/2026
Converted date: 12-Oct-26
```

Another example:

```
Enter date (dd/mm/yyyy): 05/01/2025
Converted date: 05-Jan-25
```

Note: This program checks the date's format and basic ranges, but does not validate the number of days in each month.

Question 2: Print All Substrings of a String

1. Aim

To write a C program using a separate user-defined function to print all possible non-empty substrings of a given string.

2. Algorithm

1. Start the program.

2. Read a string from the user.

3. Calculate its length using `strlen()`.

4. Use an outer loop to select the starting position of a substring.

5. Use an inner loop to select the ending position.

6. Print each substring using the appropriate length.

7. Continue until all possible substrings have been printed.

8. Stop the program.

3. C Program

```
#include <stdio.h>
#include <string.h>

#define SIZE 100

void printSubstrings(char str[]);

int main()
{
    char str[SIZE];

    printf("Enter a string: ");
    scanf("%99s", str);

    printSubstrings(str);
```

```
    return 0;
}

void printSubstrings(char str[])
{
    int i, j;
    int len = strlen(str);

    printf("All substrings: ");

    for (i = 0; i < len; i++)
    {
        for (j = i; j < len; j++)
        {
            printf("%.*s", j - i + 1, str + i);

            if (i != len - 1 || j != len - 1)
                printf(", ");
        }
    }

    printf("\n");
}
```

4. Step-by-step explanation

Step 1: Read the string

`scanf("%99s", str);`
This reads a string such as `abc`. It is suitable for a single word without spaces.

Step 2: Calculate the string length

`int len = strlen(str);`
For `abc`, the length is `3`.

Step 3: Select the starting position

`for (i = 0; i < len; i++)`
The outer loop selects each character as the starting point of a substring.

Step 4: Select the ending position

`for (j = i; j < len; j++)`
The inner loop extends the substring from its starting position to each possible ending position.

Step 5: Print the substring

```
printf("%.*s", j - i + 1, str + i);
```

- `str + i` points to the starting character.
- `j - i + 1` calculates the number of characters to print.
- `%.*s` prints exactly that number of characters.

5. Dry Run for Input `abc`

Starting index `i`	Ending index `j`	Substring
0	0	a
0	1	ab
0	2	abc
1	1	b
1	2	bc
2	2	c

6. Sample Output

```
Enter a string: abc
All substrings: a, ab, abc, b, bc, c
```

3. Combined Menu-Driven C Program

Since the question specifically asks for a menu-driven program, use the following complete program for your practical submission. It performs both operations using separate functions.

```
#include <stdio.h>
#include <string.h>

#define SIZE 100

void convertDate(char date[]);
void printSubstrings(char str[]);

int main()
{
    char date[20], str[SIZE];
```

```
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Convert date format\n");
        printf("2. Print all substrings\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");

        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter date (dd/mm/yyyy): ");
                scanf("%19s", date);
                convertDate(date);
                break;

            case 2:
                printf("Enter a string: ");
                scanf("%99s", str);
                printSubstrings(str);
                break;

            case 3:
                printf("Program exited.\n");
                break;

            default:
                printf("Invalid choice!\n");
        }

    } while (choice != 3);

    return 0;
}

void convertDate(char date[])
{
    int day, month, year;
    char *months[] = {
        "Jan", "Feb", "Mar", "Apr",
        "May", "Jun", "Jul", "Aug",
```

```
        "Sep", "Oct", "Nov", "Dec"
    };

    if (sscanf(date, "%d/%d/%d", &day, &month, &year) != 3 ||
        day < 1 || day > 31 || month < 1 || month > 12 ||
        year < 0 || year > 9999)
    {
        printf("Invalid date format!\n");
        return;
    }

    printf("Converted date: %02d-%s-%02d\n",
           day, months[month - 1], year % 100);
}

void printSubstrings(char str[])
{
    int i, j;
    int len = strlen(str);

    printf("All substrings: ");

    for (i = 0; i < len; i++)
    {
        for (j = i; j < len; j++)
        {
            printf("%.*s", j - i + 1, str + i);

            if (i != len - 1 || j != len - 1)
                printf(", ");
        }
    }

    printf("\n");
}
```

4. Sample Output of the Combined Program

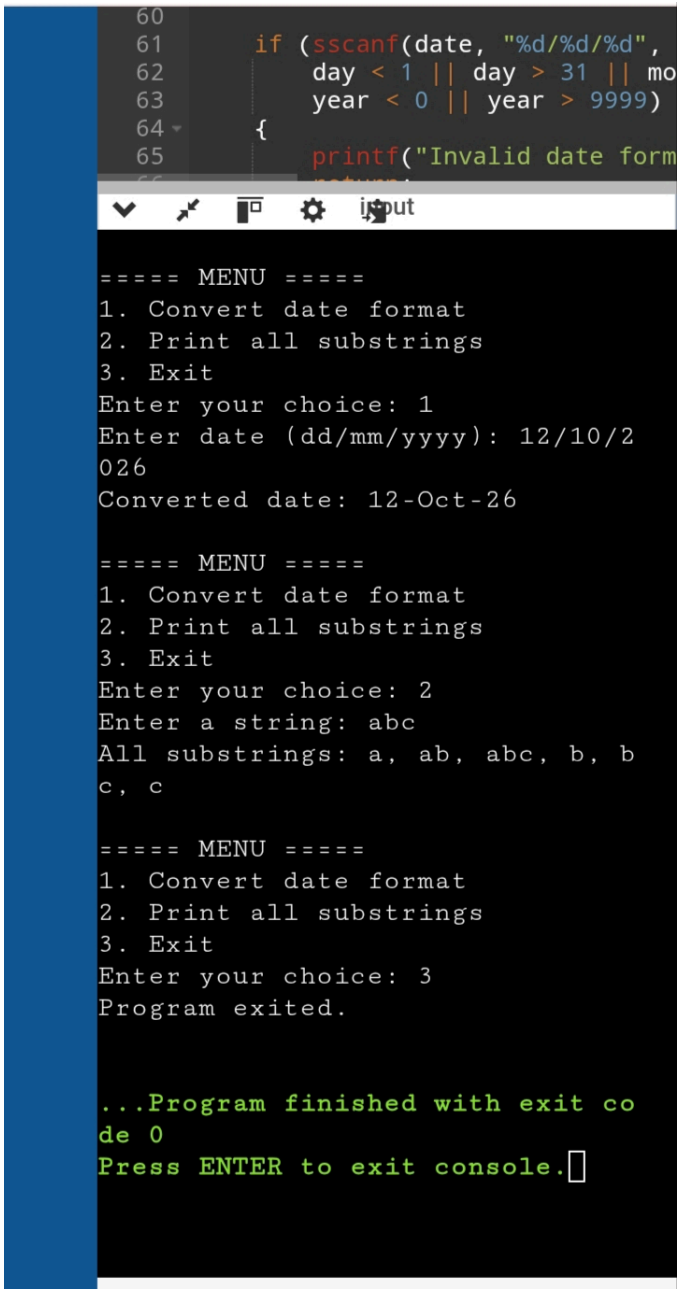
```
===== MENU =====
1. Convert date format
2. Print all substrings
3. Exit
Enter your choice: 1
Enter date (dd/mm/yyyy): 12/10/2026
Converted date: 12-Oct-26
```

```
===== MENU =====
1. Convert date format
2. Print all substrings
3. Exit
Enter your choice: 2
Enter a string: abc
All substrings: a, ab, abc, b, bc, c

===== MENU =====
1. Convert date format
2. Print all substrings
3. Exit
Enter your choice: 3
Program exited.
```

5. Result

The menu-driven C program successfully converts dates from `dd/mm/yyyy` to `dd-mmm-yy` format and prints all possible non-empty substrings of a string using separate user-defined functions.


60
61 if (sscanf(date, "%d/%d/%d",
62 day < 1 || day > 31 || mo
63 year < 0 || year > 9999)
64 {
65 printf("Invalid date form
input
===== MENU =====
1. Convert date format
2. Print all substrings
3. Exit
Enter your choice: 1
Enter date (dd/mm/yyyy): 12/10/2
026
Converted date: 12-Oct-26
===== MENU =====
1. Convert date format
2. Print all substrings
3. Exit
Enter your choice: 2
Enter a string: abc
All substrings: a, ab, abc, b, b
c, c
===== MENU =====
1. Convert date format
2. Print all substrings
3. Exit
Enter your choice: 3
Program exited.
...Program finished with exit co
de 0
Press ENTER to exit console.